

Capital One 2019 Cloud Breach

CASE STUDY

In 2019, one of the largest financial institutions in the United States suffered a catastrophic data breach — not because AWS failed, but because Capital One's own configuration choices created the vulnerability. This case study examines how five compounding failures on the *customer side* of the shared responsibility model exposed 100 million customers' data.



Background: Who, When, and What Was Lost

Capital One Financial Corporation, one of the largest banks in the United States, discovered a major breach in **July 2019**. The attacker was Paige Thompson, a former AWS engineer who identified and exploited a misconfiguration in Capital One's AWS environment beginning in **March 2019**.

Over several months, she exfiltrated a staggering volume of sensitive customer data — entirely undetected. She was eventually identified after posting about her access on a public Slack channel and on GitHub, leading to her arrest in July 2019, conviction in 2022, and subsequent sentencing.

Data Exfiltrated

100M

Customer Records

6M

Credit Card
Applications

140K

Social Security
Numbers

80K

Bank Account Numbers



The Technical Chain: Five Compounding Failures

No single vulnerability caused the breach. Five distinct failures — each within Capital One's own configuration responsibilities — combined into one catastrophic attack path. Every failure was entirely on the **customer side** of the AWS Shared Responsibility Model.



AWS's underlying infrastructure functioned exactly as designed throughout. The breach was not a platform failure — it was a configuration failure, repeated five times over.

Breaking Down Each Failure

- 1 Misconfigured WAF**

Capital One deployed ModSecurity on an EC2 instance as a Web Application Firewall. The WAF's attached IAM role granted it overly broad permissions — including the ability to **list and read S3 buckets** across the account, far beyond what a WAF operationally requires.
- 2 SSRF Vulnerability**

The application allowed an attacker to coerce the server into making requests to arbitrary URLs — a well-known vulnerability class. In AWS, SSRF is especially dangerous: the metadata service at `169.254.169.254` returns the temporary credentials assigned to the EC2 instance's IAM role.
- 3 Excessive IAM Permissions**

The IAM role attached to the WAF EC2 instance had permissions to list buckets and read objects across **many S3 buckets**, including those containing credit card application data. The principle of least privilege had not been applied.
- 4 No Detection Controls**

100 million records being read from S3 by an unusual source over several months produced **no alarm**. CloudTrail logs existed but were not actively monitored. AWS GuardDuty was not configured to detect the anomalous access pattern.
- 5 No Trust Segmentation**

The WAF's role and the data buckets lacked explicit cross-account boundaries or resource-policy controls. Even if the credentials were known to be compromised, **nothing would have blocked access** at the data layer.

How the Attack Unfolded



Discover SSRF

Attacker identifies the SSRF vulnerability in Capital One's web application



Hit Metadata Service

Exploits SSRF to query `169.254.169.254` and retrieve the WAF EC2 instance's temporary IAM credentials



Enumerate S3

Uses stolen credentials to list all accessible S3 buckets — including those holding sensitive customer PII



Exfiltrate Data

Reads and downloads 100 million customer records over several months, entirely undetected



Self-Identified

Posts about the access on Slack and GitHub; identified and arrested July 2019

The Cost of Getting It Wrong

The financial and reputational consequences were severe and wide-ranging. Beyond direct penalties, the breach triggered years of regulatory scrutiny, remediation programmes, and lasting reputational damage — a reminder that misconfiguration carries enterprise-level consequences.

Regulatory Fine

US\$80 million penalty levied by the Office of the Comptroller of the Currency (OCC) for failure to establish effective risk management controls in its cloud environment

Class Action Settlement

US\$190 million settlement of customer class action lawsuits covering over 100 million affected individuals across the United States and Canada

Remediation Costs

Significant additional undisclosed costs for forensic investigation, infrastructure remediation, enhanced monitoring, and mandatory regulatory compliance programmes



Mapping Failures to Security Principles

Every failure in this breach maps directly to foundational cloud security concepts. This is not an edge case — it is a textbook demonstration of what happens when well-understood principles are not applied consistently.

Shared Responsibility Model

Every failure was squarely on the **customer side**. AWS's substrate worked exactly as designed. The shared responsibility model is unambiguous: the provider gives you secure capabilities; using them correctly is your job. Capital One's configuration choices — not AWS — created the breach.

Cloud Architecture & Network Segmentation

Better network architecture — **VPC endpoints, bucket policies restricted to specific VPCs, and explicit deny-by-default** — could have made the WAF structurally unable to reach PII buckets regardless of IAM permissions. Defence-in-depth at the network layer was absent.

Data Protection at Rest

Customer data was stored in S3 but was readable by any properly-permissioned IAM principal. **KMS-based bucket encryption with strict key policies** would have added a critical layer: even with stolen credentials, the role could not have decrypted the data without explicit KMS key access.

Zero Trust Architecture

This is a **textbook failure of perimeter trust assumptions** inside the cloud. The WAF's IAM role was implicitly trusted for anything its policy allowed, with no continuous verification of context — access pattern, source, volume, or time. Zero trust principles applied to workload identity would have made this breach significantly harder to execute.

Discussion: What Would Have Stopped It?

GROUP DISCUSSION



Q1: The Single Best Detective Control

What single AWS-native control, if properly configured, would most likely have caught this breach *in progress*?

Model Answer: AWS GuardDuty — specifically the IAMUser/InstanceCredentialExfiltration finding type, which detects EC2 instance credentials being used from outside the account. CloudTrail combined with custom Athena queries would also have caught it. The attack pattern is clearly visible in CloudTrail logs; no one was watching.



Q2: How Did the WAF Become the Attack Vector?

Capital One had a WAF — the very tool designed to defend against application attacks. How could it become the origin of the breach?

Model Answer: The WAF held an IAM role far broader than its operational needs. A WAF needs to inspect web traffic — it has **no legitimate need** to list and read S3 buckets containing customer applications. Roles should be scoped strictly. Granting broad permissions "just in case" and never revisiting them is a common, dangerous anti-pattern.



Q3: The Pattern Across Five Cases

How does Capital One connect to the Target, Medibank, Uber, and Colonial Pipeline breaches?

Model Answer: The same root cause class: **trust granted based on context — network location, identity, role — without continuous verification**. Five industries, five countries, one pattern. An attacker who acquires a trusted identity moves freely. The architectural fix is identical in every case: zero trust applied wherever trust has been over-extended.

Key Controls That Would Have Prevented This

The breach was preventable at multiple layers. The following controls — all AWS-native and well-documented — represent the defence-in-depth posture that was absent.

Preventive Controls

- **Least-privilege IAM roles:** The WAF role should have had zero S3 permissions, or strictly scoped read-only access to a specific log bucket only
- **IMDSv2 (Instance Metadata Service v2):** Requiring session-oriented requests to the metadata service would have blocked the SSRF-based credential theft entirely
- **S3 bucket resource policies:** Explicit deny conditions restricting access to specific VPC endpoints or trusted principals, regardless of IAM role permissions
- **KMS encryption with strict key policies:** Ensuring that even a compromised role could not decrypt sensitive data without explicit key authorisation

Detective Controls

- **AWS GuardDuty:** Enabled and tuned to detect `InstanceCredentialExfiltration` findings — the exact pattern that occurred over months undetected
- **CloudTrail anomaly monitoring:** Active alerting on bulk S3 `GetObject` calls from unusual sources, combined with Athena or SIEM integration for pattern detection
- **AWS Macie:** Automated discovery and classification of sensitive data in S3, enabling targeted monitoring of buckets containing PII and financial data
- **VPC Flow Logs:** Monitoring for anomalous outbound data volumes and unexpected traffic patterns from EC2 instances handling sensitive workloads

The Takeaway: One Root Cause, One Architectural Response

Five industries. Five countries. Five different architectural eras. One root cause class: trust granted based on location or identity, without continuous verification.

The Capital One breach is not an isolated incident. It belongs to the same failure pattern as Target, Medibank, Uber, and Colonial Pipeline. In every case, an attacker who acquired or impersonated a trusted identity moved freely through the environment — because trust was implicit, static, and unchallenged.

The Customer Owns Configuration

The Shared Responsibility Model is not abstract. Capital One's breach happened entirely within the customer's zone of responsibility. Knowing where the boundary falls is the first obligation of every cloud security engineer.

Least Privilege Is Non-Negotiable

Roles should be scoped to the minimum permissions required for a specific function. Roles granted broad access "just in case" and never revisited are an open invitation for lateral movement when credentials are compromised.

Zero Trust Is the Response

Continuous verification of context — access pattern, source, volume, behaviour — applied to workload identity is what makes the difference. Perimeter trust assumptions inside the cloud are a design flaw, not a shortcut. Zero trust is the architectural answer.